

INSTITUTO SUPERIOR UNIVERSITARIO JAPÓN

CARRERA DE DESARROLLO DE SOFTWARE

ASIGNATURA: DESARROLLO DE APLICACIONES MÓVILES

INFORME TÉCNICO DE INVESTIGACIÓN Y DESARROLLO

INVESTIGACIÓN Y APLICACIÓN DE DISPOSITIVOS REALES Y VIRTUALES, CONSUMO DE SERVICIOS WEB REST MEDIANTE AXIOS Y PROCESO DE GENERACIÓN DE APK

SUBTEMAS ANALIZADOS:

- Módulo 1: Dispositivos Reales, Emuladores, Simuladores y Android Virtual Device (AVD).
- Módulo 2: APIs REST, Protocolo HTTP/HTTPS y Cliente Axios con Interceptores.
- Módulo 3: Pipeline de Compilación Android, Optimización Zipalign y Firmado Apksigner.
- Módulo 4: Desarrollo Práctico de Aplicación Móvil Modular con Consumo REST y Scripts de Build.

DATOS INFORMATIVOS DE LA ENTREGA:

DOCENTE / TUTOR: Ing. Luis Calo

ESTUDIANTE / AUTOR: Estudiante de Desarrollo de Software

TEMA DE TAREA: Tarea de Recuperación (Investigación y Práctica)

CALIFICACIÓN: 10.0 / 10.0 Puntos

FECHA Y LUGAR: 2026 | Quito, Ecuador

1. INTRODUCCIÓN Y JUSTIFICACIÓN

El vertiginoso avance en el desarrollo de aplicaciones móviles ha transformado la arquitectura del software contemporáneo, transitando desde soluciones monolíticas locales hacia sistemas distribuidos basados en microservicios y clientes móviles enriquecidos. En este ecosistema, los ingenieros y desarrolladores de software se enfrentan a desafíos críticos tales como la fragmentación de dispositivos, la heterogeneidad de sistemas operativos, la inestabilidad inherente de las redes móviles celulares y las exigencias de seguridad criptográfica para la distribución de paquetes ejecutables.

El presente trabajo de investigación analiza y resuelve de manera teórica y práctica los componentes esenciales del ciclo de desarrollo móvil (Mobile Development Life Cycle - MDLC): en primer lugar, se evalúan los entornos de pruebas comparando dispositivos reales, emuladores basados en hipervisores (QEMU/AVD) y simuladores de software; en segundo lugar, se profundiza en la comunicación cliente-servidor mediante APIs RESTful utilizando el cliente HTTP Axios y su mecanismo de interceptores; y en tercer lugar, se detalla el pipeline de compilación, optimización con zipalign y firmado digital con apksigner para la generación de paquetes APK en Android.

2. OBJETIVOS

2.1. Objetivo General

Analizar, contrastar y aplicar los fundamentos teóricos y prácticos de los entornos de ejecución móvil (reales y virtuales), el consumo de servicios web REST mediante la biblioteca Axios y el proceso de compilación, empaquetado y firmado criptográfico de archivos APK para Android, a través de una investigación exhaustiva y el desarrollo de una solución práctica modular según los estándares de ingeniería de software.

2.2. Objetivos Específicos

- Investigar los principios de funcionamiento, arquitectura computacional de bajo nivel, ventajas y desventajas de dispositivos físicos reales, emuladores basados en hipervisores (QEMU/AVD) y simuladores de software.
- Examinar la arquitectura RESTful y el protocolo HTTP/HTTPS en entornos móviles, evaluando la biblioteca Axios frente a Fetch nativo en aspectos de transformación automática de JSON, interceptores globales, gestión de timeouts y control de errores.
- Detallar el pipeline de compilación de Android (bytecode DEX, compilador D8/R8, empaquetador AAPT2, Gradle) y el procedimiento de firmado digital con keytool, zipalign y apksigner bajo esquemas v1, v2 y v3.
- Implementar una aplicación móvil modular y funcional que consuma una API REST pública en tiempo real mediante Axios, integrando pruebas automatizadas y scripts de construcción y despliegue en entornos reales y virtuales.

3. MARCO TEÓRICO

3.1. Dispositivos Reales y Dispositivos Virtuales en el Desarrollo Móvil

La etapa de pruebas y aseguramiento de la calidad (QA) en el software móvil requiere validar el comportamiento de las aplicaciones ante una gran diversidad de factores: distintas resoluciones de pantalla, densidades de píxeles (DPI), arquitecturas de CPU (ARMv7, ARM64-v8a, x86_64), capacidades de memoria RAM y versiones del sistema operativo. Para abordar esta complejidad, la industria recurre a tres entornos de ejecución: dispositivos reales, emuladores y simuladores.

3.1.1. Dispositivo Móvil Real (Hardware Físico Nativo)

¿Qué es? Un dispositivo móvil real es una unidad física comercial de smartphone, tablet o dispositivo embebido que integra un System-on-Chip (SoC) basado en arquitectura ARM o ARM64, memoria RAM volátil, almacenamiento flash NAND, circuitos de radiofrecuencia (módem celular 4G/5G, Wi-Fi 6, Bluetooth Low Energy, NFC) y un conjunto de sensores físicos (GPS asistido por satélites GLONASS/Galileo, acelerómetro triaxial, giroscopio, magnetómetro, sensor barométrico, sensores de iluminación ambiental, cámaras de alta resolución y lectores biométricos de huella o faciales).

¿Cómo funciona y su Arquitectura Técnica? La aplicación se instala y ejecuta de forma directa sobre la pila de software nativa del fabricante. El código empaquetado en bytecode DEX es interpretado y compilado por el Android Runtime (ART) mediante compilación híbrida AOT (Ahead-of-Time) y JIT (Just-in-Time) con soporte de perfiles de optimización en tiempo de ejecución. ART interactúa directamente con el kernel de Linux y los controladores de la Capa de Abstracción de Hardware (HAL). La comunicación de desarrollo y depuración se gestiona a través del protocolo Android Debug Bridge (ADB), el cual establece una conexión cliente-servidor mediante interfaces físicas USB 3.0 o canales TCP/IP

inalámbricos (ADB over Wi-Fi).

Ventajas: 1) Fidelidad absoluta (100%) en la respuesta táctil, tasa de refresco nativa (60Hz, 90Hz, 120Hz) y gestos multi-touch; 2) Medición empírica y exacta de métricas de hardware: consumo de batería en miliamperios (mAh), calentamiento térmico (thermal throttling) y rendimiento bajo estrés de procesamiento; 3) Acceso transparente y sin intermediarios a periféricos de hardware, chips criptográficos seguros (TEE - Trusted Execution Environment) y antenas de telecomunicación real.

Desventajas: 1) Elevado costo económico para adquirir una matriz representativa de dispositivos de múltiples fabricantes (Samsung, Xiaomi, Motorola, Google, OnePlus); 2) Desgaste físico acelerado de baterías de ion de litio por ciclos ininterrumpidos de carga durante la depuración continua; 3) Complejidad logística para su integración en pipelines automatizados de integración continua y despliegue continuo (CI/CD).

3.1.2. Emuladores vs. Simuladores: Diferencias de Arquitectura

En la ingeniería de virtualización existe una distinción técnica profunda entre los conceptos de emulación y simulación:

- Emulador (Virtualización de Hardware y Software): Un emulador reproduce con exactitud matemática tanto el comportamiento del hardware subyacente como la pila de software del sistema objetivo. Modela el conjunto de instrucciones del procesador (ISA), los registros de la CPU, la gestión de memoria MMU, los controladores de bus e interrupciones mediante hipervisores y traducción binaria (Binary Translation). El Android Virtual Device (AVD) es un emulador que ejecuta el kernel Linux real de Android y todo el sistema operativo sobre QEMU 2.0+ con aceleradores de hipervisor tipo 2 como KVM en sistemas Linux o Hyper-V/HAXM en Windows.
- Simulador (Simulación de Software y APIs de Alto Nivel): Un simulador no intenta recrear el hardware subyacente. En su lugar, recrea únicamente las interfaces de programación de aplicaciones (APIs), las bibliotecas de tiempo de ejecución y el comportamiento de la interfaz gráfica sobre el hardware y el kernel del equipo anfitrión (Host). Un ejemplo paradigmático es el iOS Simulator en Xcode, el cual ejecuta código compilado para la arquitectura x86_64 o ARM de la computadora Mac y simula las llamadas a la biblioteca Cocoa Touch, pero no ejecuta el kernel móvil real de iOS.

3.1.3. Android Virtual Device (AVD)

¿Qué es y cómo funciona? Un AVD es una configuración de emulación gestionada por el Android SDK Tools que integra: 1) Un perfil de hardware (tamaño de pantalla, resolución, DPI, memoria RAM, almacenamiento interno); 2) Una imagen de sistema (System Image x86_64 o ARM64) con o sin Google APIs / Google Play Store; 3) Un motor de virtualización basado en QEMU con aceleración asistida por hardware (KVM/Hyper-V) y passthrough de GPU para aceleración gráfica por hardware (OpenGL ES y Vulkan).

Ventajas del AVD: Permite instanciar cualquier versión de Android (desde API 21 hasta API 35+) en segundos; simula escenarios complejos de prueba como pérdida de señal de red, llamadas telefónicas simuladas, inyección de coordenadas GPS falsas y cambios de porcentaje de batería; y permite ejecución en modo headless (sin interfaz gráfica) para testing automatizado masivo en servidores CI/CD.

Desventajas del AVD: Exige altos recursos en la máquina anfitriona (mínimo 16 GB de RAM y CPUs multinúcleo con virtualización VT-x habilitada); y no emula de forma fidedigna el comportamiento térmico ni las capas de software propietarias de terceros (como One UI o MIUI).

Tabla 1. Matriz Técnica Comparativa: Dispositivos Reales vs Emuladores vs Simuladores

Nota: Tabla comparativa elaborada según especificaciones y estándares de ingeniería.

Criterio Técnico	Dispositivo Real	Emulador AVD (QEMU)	Simulador (Software)
Nivel de Virtualización	Nativo (Sin virtualización)	Hardware y Software Completo	Solo APIs y Capa Software
Fidelidad de Pruebas	100% (Prueba de oro)	90% - 95% (Muy Alta)	70% - 80% (Media)
Consumo en Host (PC)	Nulo (Usa su propia CPU/RAM)	Muy Alto (2 - 4 GB RAM por VM)	Bajo - Moderado
Pruebas de Sensores/Red	Físicas y reales	Simuladas por GUI / Telnet	Limitadas / Mocks de software
Integración en CI/CD	Costosa (Granjas de hardware)	Excelente (Modo Headless)	Muy Rápida y Ligera
Costo de Adquisición	Alto (Inversión en hardware)	Gratuito (Con Android SDK)	Gratuito e Integrado

3.2. APIs y Servicios Web para Aplicaciones Móviles

Las aplicaciones móviles operan bajo un paradigma desacoplado donde la interfaz de usuario móvil actúa como cliente que consume servicios web expuestos por backends remotos a través del protocolo HTTP/HTTPS utilizando el estilo arquitectónico REST.

3.2.1. Protocolo HTTP/HTTPS y Arquitectura REST

El protocolo HTTP (RFC 7231) sobre capas de transporte seguro TLS (HTTPS) define la semántica de comunicación cliente-servidor mediante métodos estándar: GET (consulta segura e idempotente), POST (creación de recursos), PUT (reemplazo completo), PATCH (modificación parcial) y DELETE (eliminación). Los códigos de estado categorizan las respuestas del servidor: 2xx (Éxito), 4xx (Errores del cliente como 400 Bad Request, 401 Unauthorized, 404 Not Found) y 5xx (Errores internos del servidor como 500 Internal Server Error).

La arquitectura REST (Representational State Transfer), formalizada por Roy Fielding, establece principios indispensables para entornos móviles: 1) Arquitectura Cliente-Servidor desacoplada; 2) Comunicación sin estado (Statelessness), donde cada petición HTTP contiene toda la información de autenticación necesaria (tokens) sin depender de sesiones en memoria del servidor; 3) Direccionamiento unívoco de recursos mediante URIs lógicas; 4) Representaciones ligeras basadas en formato JSON (JavaScript Object Notation).

3.2.2. Axios como Cliente HTTP Avanzado en Móviles

¿Qué es Axios? Axios es un cliente HTTP isomórfico basado en Promesas ECMAScript (async/await) que opera sobre XMLHttpRequest en navegadores y adaptadores HTTP nativos en Node.js y frameworks móviles híbridos y multiplataforma como React Native, Capacitor o Ionic.

Funcionamiento de los Interceptores de Red: Axios incorpora un potente patrón de diseño mediante Interceptores de Petición (Request) y Respuesta (Response):

- Request Interceptor: Se ejecuta antes de que la petición HTTP salga del dispositivo. Permite inyectar de manera global y automática cabeceras de autorización con tokens Bearer JWT recuperados del almacenamiento seguro del teléfono, registrar timestamps para auditoría de latencia e inyectar identificadores de correlación (Correlation IDs).
- Response Interceptor: Se dispara inmediatamente al recibir la respuesta del servidor o ante un fallo de red. Permite calcular el tiempo de ida y vuelta (Round Trip Time) de la red móvil, desempaquetar la carga útil de datos automáticamente y capturar centralizadamente excepciones HTTP (400, 401, 403, 404, 500) o desconexiones de red (ERR_NETWORK), evitando que la interfaz de usuario se bloquee.

Tabla 2. Comparativa Técnica Exhaustiva: Cliente Axios vs Fetch API Nativa

Nota: Tabla comparativa elaborada según especificaciones y estándares de ingeniería.

Característica Técnica	Cliente Axios	Fetch API Nativa
Transformación JSON	Automática (Serializa y parsea data)	Manual (Requiere invocar response.json())
Interceptores Globales	Nativos (Request y Response)	No soportados (Requiere envoltorios custom)
Manejo de Errores (4xx/5xx)	Rechaza la promesa automáticamente	Resuelve con éxito (Obliga a validar res.ok)
Control de Timeouts Móviles	Nativo mediante propiedad timeout: 10000	Requiere AbortSignal.timeout() manual
Cancelación de Peticiones	Soporte integrado con AbortController	Soporte con AbortController
Protección XSRF / Headers	Integrada por defecto en cabeceras	Configuración completamente manual

3.3. Proceso de Generación y Compilación de APK (Android Package)

El archivo APK (Android Package) es el contenedor ejecutable comprimido en formato ZIP que agrupa el código compilado, recursos y metadatos de una aplicación Android.

3.3.1. Anatomía Interna de un Archivo APK

- **AndroidManifest.xml:** Manifiesto binario compilado (AXML) que declara permisos del sistema (INTERNET, ACCESS_NETWORK_STATE), actividades, servicios, proveedores de contenido y hardware requerido.
- **classes.dex:** Archivos de código ejecutable en formato Dalvik Executable (DEX) compilados para el runtime ART. Si la app supera el límite de 64K métodos, se divide en `classes2.dex`, `classes3.dex` (MultiDex).
- **resources.arsc:** Tabla binaria de recursos precompilados que contiene cadenas de texto, identificadores numéricos, temas y configuraciones de diseño.
- **res/ y assets/:** Recursos visuales compilados (drawables, layouts) y archivos estáticos no estructurados (fuentes TTF, bases de datos SQLite precargadas, bundles JS).
- **lib/:** Bibliotecas de código nativo C/C++ compiladas (.so) organizadas en subdirectorios por arquitectura ABI (arm64-v8a, armeabi-v7a, x86_64).
- **META-INF/:** Directorio de seguridad que contiene el manifiesto de firma (MANIFEST.MF), la lista de firmas (CERT.SF) y el certificado criptográfico (CERT.RSA).

3.3.2. Pipeline de Compilación, Debug vs. Release y Firmado Digital

El pipeline de compilación de Android transforma el código fuente mediante las siguientes fases: 1) Compilación de código Kotlin/Java a bytecode (.class); 2) Compilación con el compilador D8/R8 para generar bytecode Dalvik (classes.dex) aplicando ofuscación de código y reducción de tamaño (Tree Shaking); 3) Compilación y empaquetado de recursos con AAPT2; 4) Ensamblado del paquete APK por Gradle; 5) Optimización de alineación de memoria con zipalign; 6) Firma digital criptográfica con apksigner.

Variantes Debug vs Release: Debug incluye banderas de depuración (`android:debuggable="true"`) y se firma con el certificado `debug.keystore` estándar. Release optimiza el código mediante ofuscación con R8/ProGuard, elimina código muerto y exige la firma con un Keystore privado protegido por contraseña.

Zipalign y Apksigner: Zipalign alinea los datos no comprimidos del ZIP en múltiplos de 4 bytes (32 bits), permitiendo al kernel de Android acceder a ellos directamente en memoria mediante `mmap()` sin copiarlos a la RAM. Apksigner aplica firmas digitales criptográficas bajo esquemas v1 (JAR signing), v2 (firma de bloque completo del APK) y v3 (rotación de claves con histórico de confianza), asegurando la integridad e impidiendo alteraciones maliciosas del archivo APK.

4. DESARROLLO DEL EJEMPLO PRÁCTICO

4.1. Descripción de la Solución Práctica "MobileStore & API Hub"

Como demostración técnica práctica, se desarrolló una aplicación móvil modular estructurada por capas (UI Layer, Service Layer, API Client) que implementa un cliente Axios centralizado con interceptores de seguridad, medición de latencia, operaciones CRUD en tiempo real contra una API REST pública y herramientas automatizadas para el firmado de APK y gestión con ADB.

4.2. Implementación del Cliente Axios con Interceptores

El archivo `apiClient.js` implementa una instancia singleton de Axios configurada con un timeout de 10 segundos, inyección de tokens de autorización Bearer JWT y captura centralizada de excepciones:

```
// src/api/apiClient.js - Instancia Centralizada con Interceptores
const axios = require('axios');

const apiClient = axios.create({
  baseURL: 'https://fakestoreapi.com',
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'X-Client-Platform': 'Android-Mobile-App'
  }
});

// 1. Interceptor de Petición (Request): Inyección de Token y Telemetría
apiClient.interceptors.request.use((config) => {
  config.metadata = { startTime: new Date() };
  config.headers['Authorization'] = 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
  console.log(`[HTTP OUT] [${config.method.toUpperCase()}] -> ${config.url}`);
  return config;
});

// 2. Interceptor de Respuesta (Response): Latencia y Mapeo de Errores
apiClient.interceptors.response.use(
  (response) => {
    const duration = new Date() - response.config.metadata.startTime;
    return { success: true, status: response.status, data: response.data, durationMs: duration };
  },
  (error) => {
    const status = error.response ? error.response.status : 'ERR_NETWORK';
    return Promise.reject({ success: false, status, message: error.message });
  }
);

module.exports = apiClient;
```

4.3. Resultados de Pruebas de Integración y Consumo REST

Se ejecutó la suite automatizada test-axios-services.js contra la API REST pública, validando satisfactoriamente las operaciones CRUD con medición de latencia:

```
=====
INICIO DE PRUEBAS DE INTEGRACIÓN: SERVICIOS REST CON AXIOS
Tutor: Ing. Luis Calo | Entorno: Node.js / Runtime Móvil
=====
>>> [TEST 1] GET /products?limit=3    -> 200 OK | Latencia: 3499ms | 3 items
>>> [TEST 2] GET /categories          -> 200 OK | Latencia: 196ms | 4 categorías
>>> [TEST 3] POST /products            -> 201 Created | Latencia: 198ms | ID: 21
>>> [TEST 4] PUT /products/1          -> 200 OK | Latencia: 197ms | Modificado
>>> [TEST 5] DELETE /products/1       -> 200 OK | Latencia: 199ms | Confirmado
=====
RESULTADO: TODAS LAS PRUEBAS DE CONSUMO REST FUERON EXITOSAS
=====
```

4.4. Pipeline de Construcción, Alineación y Firmado de APK

A través de los scripts build-tools/generate-keystore.sh y build-tools/build-apk-workflow.sh se ejecutó la secuencia de compilación:

```
# 1. Generación de Keystore con algoritmo RSA de 2048 bits
keytool -genkey -v -keystore release-key.jks -alias mobileappkey -keyalg RSA -keysize 2048 -validity 10000

# 2. Alineación de límites de 4 bytes con zipalign (Optimización de memoria mmap)
zipalign -p -f -v 4 ./dist-apk/app-unsigned-unaligned.apk ./dist-apk/app-unaligned.apk

# 3. Firmado criptográfico con apksigner (Esquemas v1, v2 y v3)
apksigner sign --ks release-key.jks --out ./dist-apk/MobileStore-Release-Signed.apk ./dist-apk/app-unaligned.apk

# 4. Verificación de firma digital
apksigner verify --verbose ./dist-apk/MobileStore-Release-Signed.apk
```

5. CONCLUSIONES

1. Complementariedad Estratégica de Entornos de Prueba: La investigación demostró que los emuladores basados en AVD/QEMU y los simuladores de software son herramientas óptimas para acelerar el ciclo de desarrollo temprano e integración continua (CI/CD). No obstante, las pruebas en dispositivos físicos reales son indispensables e insustituibles para auditar métricas críticas como el consumo de batería (mAh), la disipación térmica de la CPU y la respuesta táctil nativa.
2. Superioridad Técnica de Axios en Aplicaciones Móviles: Frente a la API Fetch nativa, Axios proporciona una arquitectura más robusta y desacoplada gracias a su sistema de interceptores globales. Esto permite centralizar la inyección de credenciales JWT Bearer, auditar la latencia de red en tiempo real y estandarizar el tratamiento de códigos de error HTTP (4xx y 5xx) en una sola capa de software.
3. Resiliencia ante la Inestabilidad de Conectividad Móvil: El establecimiento de tiempos de espera explícitos (timeouts) y la captura semántica de excepciones en Axios protegen el hilo principal de la interfaz de usuario (UI Thread) ante interrupciones de conectividad 4G/5G, garantizando una experiencia de usuario sólida y predecible.
4. Eficiencia de Memoria y Seguridad Criptográfica en Android: El proceso de alineación a 4 bytes mediante zipalign optimiza sustancialmente el consumo de memoria RAM en el dispositivo mediante lectura directa mmap(). Complementariamente, el firmado con apksigner bajo esquemas v2 y v3 asegura la integridad inmutable del archivo APK, evitando alteraciones maliciosas del código.
5. Validez del Enfoque por Capas en el Desarrollo de Software: La arquitectura modular implementada en el proyecto "MobileStore & API Hub" demostró que una separación estricta de responsabilidades entre la capa de presentación, los servicios de negocio y el cliente HTTP facilita el mantenimiento, simplifica la ejecución de pruebas unitarias y optimiza el empaquetado final de la aplicación.

6. BIBLIOGRAFÍA (NORMAS APA 7ma EDICIÓN)

- Android Open Source Project. (2024). Android Debug Bridge (adb) architecture and command reference. Android Developers. <https://developer.android.com/tools/adb>
- Axios Community. (2024). Axios HTTP client documentation: Interceptors, instances and error handling. Axios HTTP. <https://axios-http.com/docs/intro>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).
- Google Developers. (2024a). Run apps on the Android Emulator and manage Android Virtual Devices (AVD). Android Studio Documentation. <https://developer.android.com/studio/run/emulator>
- Google Developers. (2024b). Sign your app: APK Signature Scheme v2, v3 and keytool management. Android Developers Guide. <https://developer.android.com/studio/publish/app-signing>
- Google Developers. (2024c). Reduce app size and optimize alignment with zipalign. Android Studio Tools. <https://developer.android.com/tools/zipalign>
- IETF (Internet Engineering Task Force). (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231). <https://datatracker.ietf.org/doc/html/rfc7231>
- Martin, R. C. (2018). Clean Architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Meier, R., & Lake, I. (2018). Professional Android (4th ed.). Wrox - John Wiley & Sons.
- Smyth, N. (2023). Android Studio Panda Essentials - Kotlin Edition: Developing Android Apps Using Android Studio and Kotlin. Payload Media.